

CAD-ANALYZER

Fichas de Extração — VIGAS

■ Esta ficha responde: dado um texto/linha do DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_VIGA · padrões · balanço BA*/VB*
2	Geometria LV vs FV	LINE entities · diagrama corte + planta
3	Balanço	BA*/VB* → apoio_fim="" + diagrama
4	Dimensões b/h	RE_DIM · RE_DIM_BH · convenção b menor que h
5	Schema JSON Completo	FichaFase3Viga · tramos · apoios
6	Layers Canônicos	fundo · Escoras · GARFOS · aliases
7	Confidence e Fallbacks	fórmula · cadeia de 6 níveis
8	Matriz de Decisão	balanço · sem dimensão · layer desconhecido
9	Exemplos Reais	V101 · BA-5 (balanço)
10	Pipeline Completo	fluxo E2E · checklist 12 passos

1

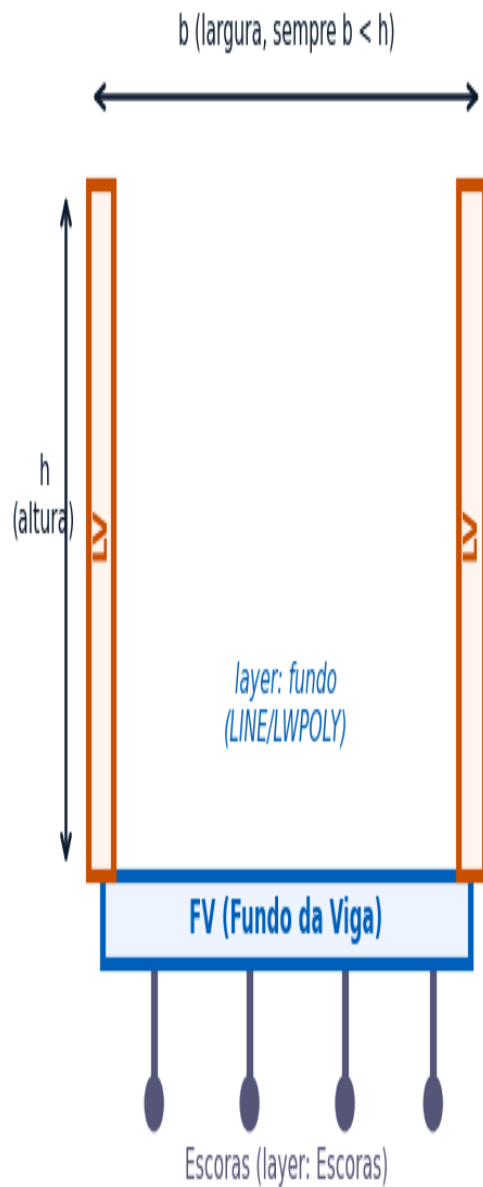
Identificação — RE_VIGA

```
RE_VIGA = re.compile(
    r'^(\V|BA|VB|VT|VC)\.?-?\d+([A-Z]|\./\d+)?',
    re.IGNORECASE
)
# V=normal BA*=balanço VB*=borda VT*=topo VC*=coroamento
```

Texto DXF	Casou?	Tipo	Obs
V101	✓	Normal	2 apoios esperados
V101A	✓	Normal	Sufixo letra
BA-5	✓	Balanço	apoio_fim="" é CORRETO
VB3	✓	Borda balanço	apoio_fim="" é CORRETO
VT-2	✓	Topo	Travamento
VC1	✓	Coroamento	—
L5	✗	—	É laje
P3	✗	—	É pilar

■ LV = painéis laterais (layer Painéis). FV = prancha de fundo (layer fundo). Distinção CRÍTICA para cálculo de material.

Componentes da Viga (Vista Frontal)



Vista Superior – Apoios e Comprimento

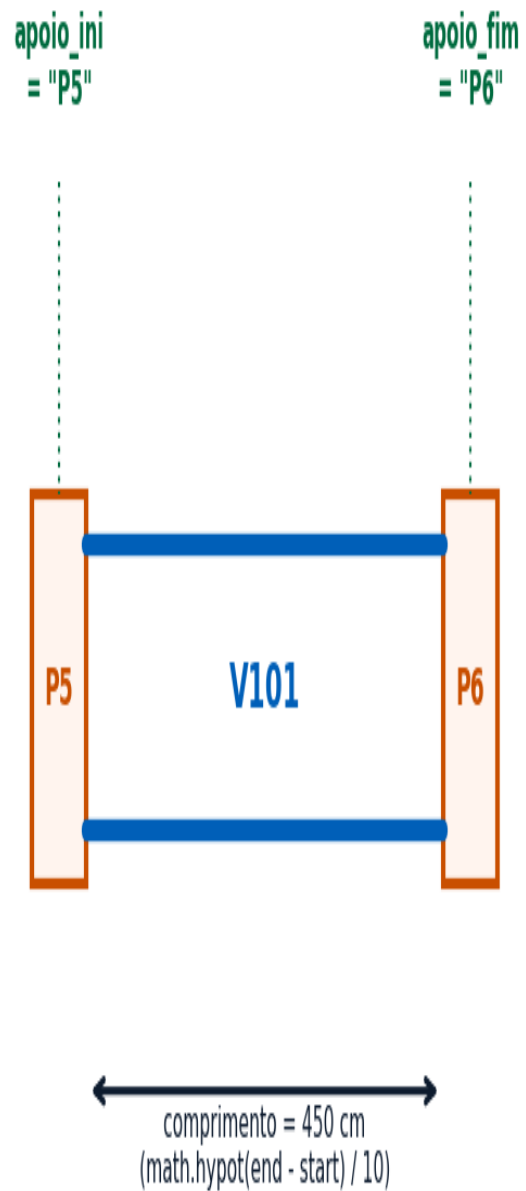


Figura 1 — Corte transversal da viga (LV/FV/Escoras) e vista superior com apoios

```
# Acesso às LINEs de fundo (FV):
for e in msp.query('LINE'):
    if normalize_layer(e.dxf.layer) not in ('FUNDO', 'FUNDOS'): continue
    start = (float(e.dxf.start.x), float(e.dxf.start.y))
    end   = (float(e.dxf.end.x),   float(e.dxf.end.y))
    comp_cm = math.hypot(end[0]-start[0], end[1]-start[1]) / 10
    fv_candidates.append({'start':start, 'end':end, 'comp':comp_cm})
```

Viga Normal vs Balanço (BA*/VB*)

Viga Normal V101

Viga Balanço BA-5

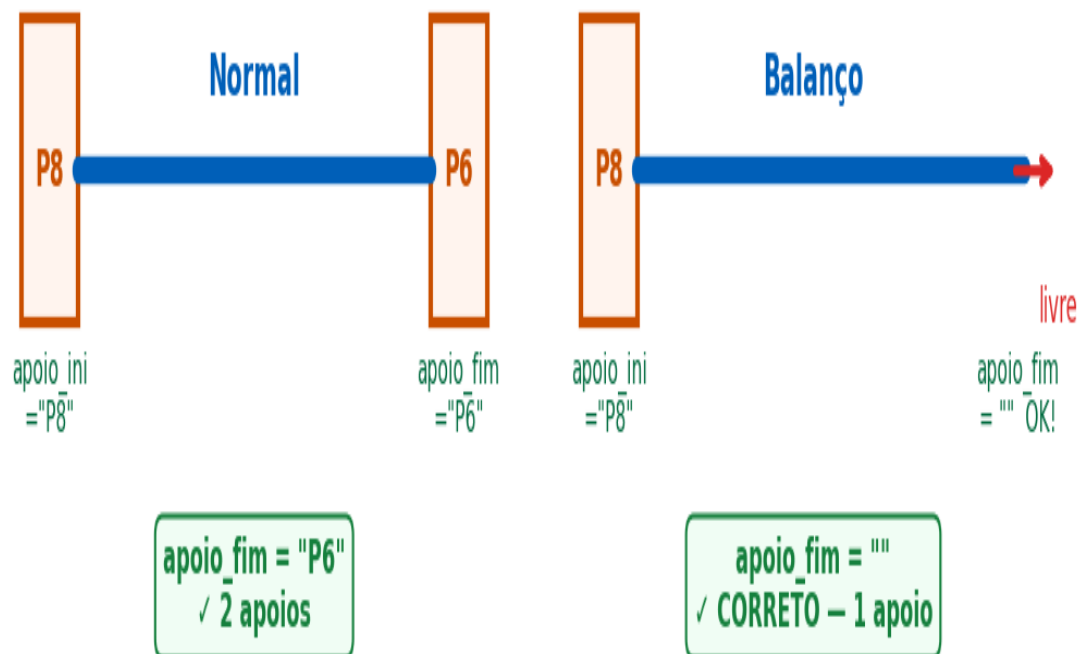


Figura 2 — Viga normal (2 apoios) vs viga em balanço BA*/VB* (1 apoio, apoio_fim="")

✓ BA* ou VB* com apoio_fim="" NÃO é erro. É o comportamento correto e esperado.

```
# Lógica de detecção:
is_balanco = bool(re.match(r'^(BA|VB)', codigo, re.I))

tramo = {
    'apoio_ini': 'P8',    # sempre preenchido
    'apoio_fim': '',      # '' se é balanço – CORRETO
    'comprimento': 220.0
}
# is_balanco=True → NÃO penalizar confidence por apoio_fim vazio
```

```

RE_DIM    = re.compile(r'(\d{1,3})\s*[xX*\/]\s*(\d{1,3})')
RE_DIM_BH = re.compile(r'b\s*=\s*(\d{1,3}).*?h\s*=\s*(\d{1,3})', re.I)

# CONVENÇÃO: b (largura) < h (altura)
largura = min(a, b) # cm - b
altura  = max(a, b) # cm - h

```

Texto DXF	largura b	altura h	Obs
"20x50"	20 cm	50 cm	b < h ✓
"50x20"	20 cm	50 cm	Reordenado: b=min, h=max
"b=20 h=50"	20 cm	50 cm	RE_DIM_BH

■ Sem dimensão em VIGA_SEARCH_RADIUS=1200mm: largura=0, altura=0, confidence == 0.30

```

{
  "codigo": "V101",
  "pavimento": "1_PAVIMENTO",  "obra_nome": "ALIMONTI",

  "largura": 20.0,                # cm – b
  "altura": 50.0,                 # cm – h

  "tramos": [{
    "apoio_ini": "P5",            # pilar inicial (extremidade start)
    "apoio_fim": "P6",           # pilar final (" se BA*/VB*")
    "comprimento": 450.0,        # cm
    "nivel": 2.80                # m
  }],

  "fv_segs": [{"start": [15000, 10000], "end": [19500, 10000]}],

  "escoras": 12,                  # COUNT LINES layer Escoras
  "garfos": 6,                    # COUNT INSERTs layer GARFOS

  "confidence": 0.90,  "revisado": false
}

```

Campo	Origem DXF
codigo	TEXT/MTEXT → RE_VIGA
largura / altura	RE_DIM/RE_DIM_BH ≤ 1200mm → b=min, h=max
tramos.apoio_ini/fim	Extremidades da LINE de fundo → pilar mais próximo ≤ 1200mm
tramos.comprimento	math.hypot(end – start) em mm → / 10 para cm
fv_segs	LINE layer fundo: start + end em mm (coordenadas DXF)
escoras	COUNT(LINE layer Escoras) próximas à viga
garfos	COUNT(INSERT layer GARFOS) próximos à viga

Canônico	Aliases reais	Entidade	Uso
BEAM_BOTTOM	fundo, FUNDOS, "Fundo da Viga"	LINE/LWPOLY	FV — comprimento da viga
SHORING	Escoras, "Escora de Viga"	LINE/INSERT	Apoio vertical
FORK_METAL	GARFOS	INSERT/LINE	Garfos HT20CT
CLAMP_METAL	presilha, Presilha, PRESILHA	LWPOLY/LINE	Presilhas metálicas
ANCHOR_BAR_LV	"BARRA DE ANCORAGEM"	LINE	Barras (≠ PL: "BARRA ANCORAGEM")
SPACER	Forcador	INSERT/LINE	Espaçadores
BATTEN_BEAM	barrote	LINE/LWPOLY	Barrotes da viga
ELEMENT_LABEL	NOMENCLATURA, texto	TEXT/MTEXT	Textos ID (V101)

Nível	Condição	Confidence
1	Texto em NOMENCLATURA → LINE em fundo/Painéis $\leq 1200\text{mm}$	raio_score
2	Texto → LINE em qualquer layer (não fundo/Painéis)	raio_score – 0.15
3	Texto + RE_DIM → dimensões extraídas	+0 (esperado)
4	Texto + RE_DIM_BH (b=20 h=50)	+0 (aceito)
5	Texto sem dimensão $\leq 1200\text{mm}$	conf – 0.30
6	BA*/VB* com apoio_fim=""	OK — balanço não penaliza

Situação	Ação	Confidence
Viga BA*/VB* (balanço)	apoio_fim="" — correto	sem penalidade
apoio_ini="" (não é balanço)	Alerta: viga sem apoio inicial	conf – 0.20
comprimento > 1500cm	Alerta: revisar	log "comprimento suspeito"
Largura menor que altura	OK — convenção b menor que h	sem penalidade
Nenhuma LINE de fundo próxima	fv_segs=[], comprimento estimado	conf – 0.25
2 LINES competindo	Vence a mais próxima ao texto	log ambas

Exemplo A — V101

```
# DXF:
TEXT layer='NOMENCLATURA' text='V101' insert=(17000,10000)
TEXT layer='NOMENCLATURA' text='20x50' insert=(17100,10050)
LINE layer='fundo' start=(15000,10000) end=(19500,10000)

# JSON: {"codigo":"V101","largura":20.0,"altura":50.0,
#       "tramos":[{"apoio_ini":"P5","apoio_fim":"P6","comprimento":450.0}],
#       "fv_segs":[{"start":[15000,10000],"end":[19500,10000]}],
#       "confidence":0.90}
```

Exemplo B — BA-5 (viga em balanço)

```
# BA* → apoio_fim='' é CORRETO (sem penalidade de confidence)
# {"codigo":"BA-5","tramos":[{"apoio_ini":"P8","apoio_fim":"","
#       "comprimento":220.0}], "confidence":0.88}
```

Passo	Ação	Detalhe
1	Carregar DXF	ezdxf.readfile(path) → msp
2	normalize_layer()	NFKD → ASCII → UPPER
3	Coletar textos RE_VIGA	TEXT/MTEXT → filtrar → lista vigas_txt
4	Coletar LINES de fundo	layer FUNDO/FUNDOS → lista fv_lines
5	Associar texto→LINE	TextAssociator: raio 1200mm → score → par
6	Extrair b/h	RE_DIM/RE_DIM_BH em 1200mm → b=min, h=max
7	Associar apoios	Extremidades da LINE → pilares_dict → apoio_ini/fim
8	Detectar balanço	BA*/VB* → apoio_fim="" é correto
9	Contar escoras/garfos	COUNT LINES Escoras + INSERTs GARFOS
10	Calcular confidence	raio_score – penalidades → threshold
11	Salvar JSON	FichaFase3Viga → JSON por obra/pavimento

Consulta 1: "Viga baldrame 14x40 comprimento 320cm"

Parâmetro	Valor
b	14.0
h	40.0
comprimento	320.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.603 !	Obra_TREINO_10	V40A	REVISAR	ID: V40A Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.601 !	Obra_TREINO_10	V40	REVISAR	ID: V40 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.591 !	Obra_TREINO_16	V40	REVISAR	ID: V40 Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.603 — elemento incomum. Revisão recomendada.

Consulta 2: "Viga de fôrma 20x60 comprimento 600cm"

Parâmetro	Valor
b	20.0
h	60.0
comprimento	600.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.559 !	Obra_TREINO_10	V40A	REVISAR	ID: V40A Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.550 !	Obra_TREINO_16	V10A	REVISAR	ID: V10A Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.550 !	Obra_TREINO_10	V50	REVISAR	ID: V50 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.559 — elemento incomum. Revisão recomendada.

Consulta 3: "Viga invertida 14x25 comprimento 180cm"

Parâmetro	Valor
b	14.0
h	25.0
comprimento	180.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.617 !	Obra_TREINO_16	V24	REVISAR	ID: V24 Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.616 !	Obra_TREINO_13	V1514	REVISAR	ID: V1514 Obra: Obra_TREINO_13 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.616 !	Obra_TREINO_21	V24	REVISAR	ID: V24 Obra: Obra_TREINO_21 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.617 — elemento incomum. Revisão recomendada.

Thresholds de Decisão RAG

Similarity	Ação	Comportamento do Pipeline
≥ 0.85	ACEITAR	Elemento típico — gerar DXF sem restrições
≥ 0.65	ACEITAR_COM_AVISO	Elemento similar — alertar revisor, prosseguir

Similarity	Ação	Comportamento do Pipeline
≥ 0.40	REVISAR	Elemento incomum — suspender até revisão humana
< 0.40	REJEITAR	Elemento anômalo — bloquear geração DXF

Fórmula de Anomaly Score

```
anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty

dim_penalty:
    0.0 → Todas as dimensões OK (dentro dos limites calibrados)
    0.5 → Pelo menos 1 AVISO dimensional
    1.0 → Pelo menos 1 CRÍTICO dimensional

Categorias:
    NORMAL   (0.00-0.30) → Elemento típico do corpus
    INCOMUM  (0.30-0.55) → Aceitar com atenção
    SUSPEITO (0.55-0.75) → Revisar antes de gerar DXF
    ANOMALO  (0.75-1.00) → Bloquear geração
```

Integração no Pipeline

```
# Em robot_integration.py (antes de engine.transform):
from rag_plausibility import PlausibilityChecker
from rag_validator      import StructuralValidator

_plaus = PlausibilityChecker()
_val    = StructuralValidator()

# 1. Validação dimensional (hard limits – determinístico)
val = _val.validate(tipo, eid, entity_data, obra)
if val.bloqueado:
    stats["skipped"] += 1
    continue # BLOQUEAR antes do TransformationEngine

# 2. Plausibilidade semântica (RAG)
plaus = _plaus.check(eid, tipo, entity_data, obra)
if plaus.acao in ("REVISAR", "REJEITAR"):
    entity_data["_rag_nota"] = plaus.nota_rag # enriquecer
```